Procesos de Desarrollo de Software

CLASE 12 – PATRONES DE DISEÑO

Agenda de la clase

- Observer

Observer

Es un patrón de **Comportamiento.**

¿Qué hace?

- Permite notificar a los interesados de cierto sujeto los eventos que a éste le ocurren y que ellos actúen en consecuencia.
- Desacopla las acciones que se ejecutan de los eventos que las activan.

Se **sugiere** su utilización **cuando**:

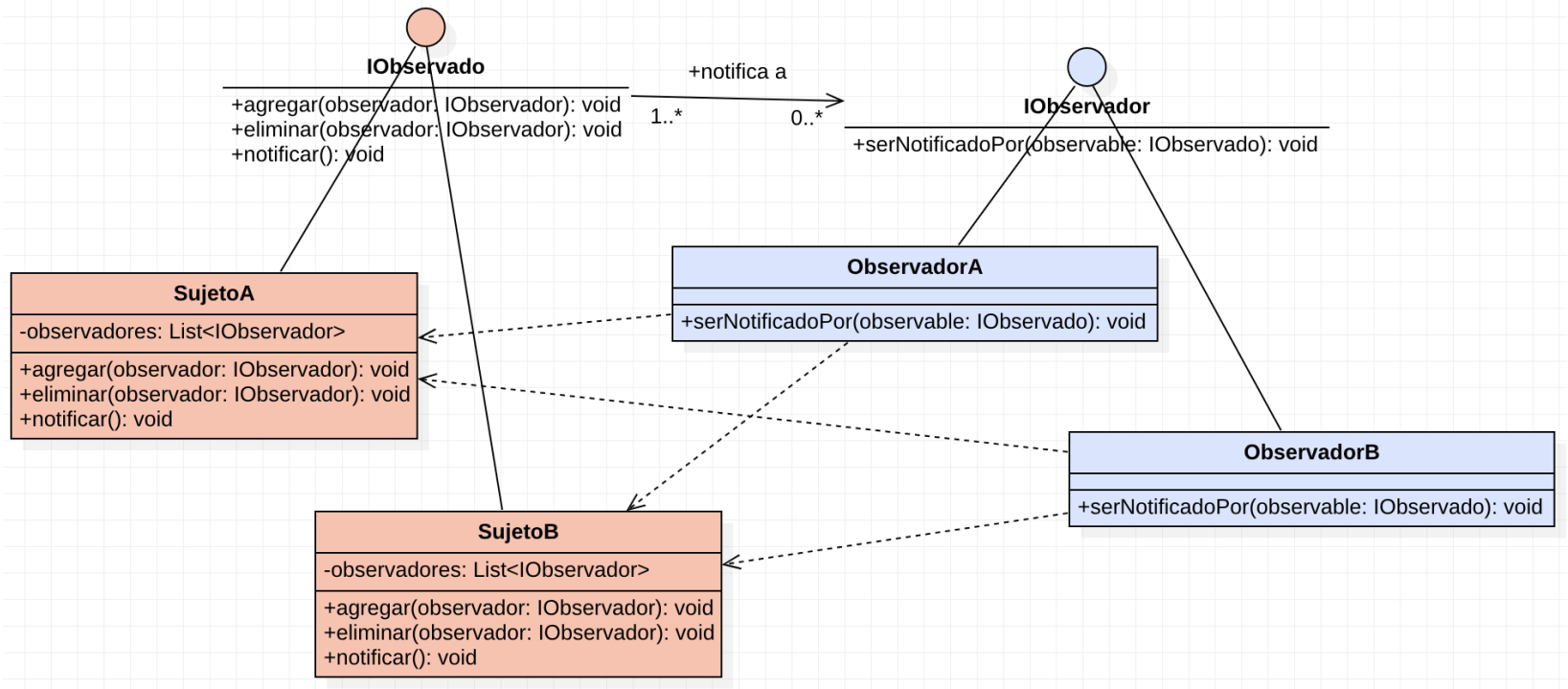
- Se requiere disparar acciones de diferentes naturalezas frente a la ocurrencia de un evento en algún objeto.
- Se requiere notificar/avisar a algunos interesados de un objeto sobre la ocurrencia de algún evento y no interesa saber quiénes son y/o cuántos son.

Componentes

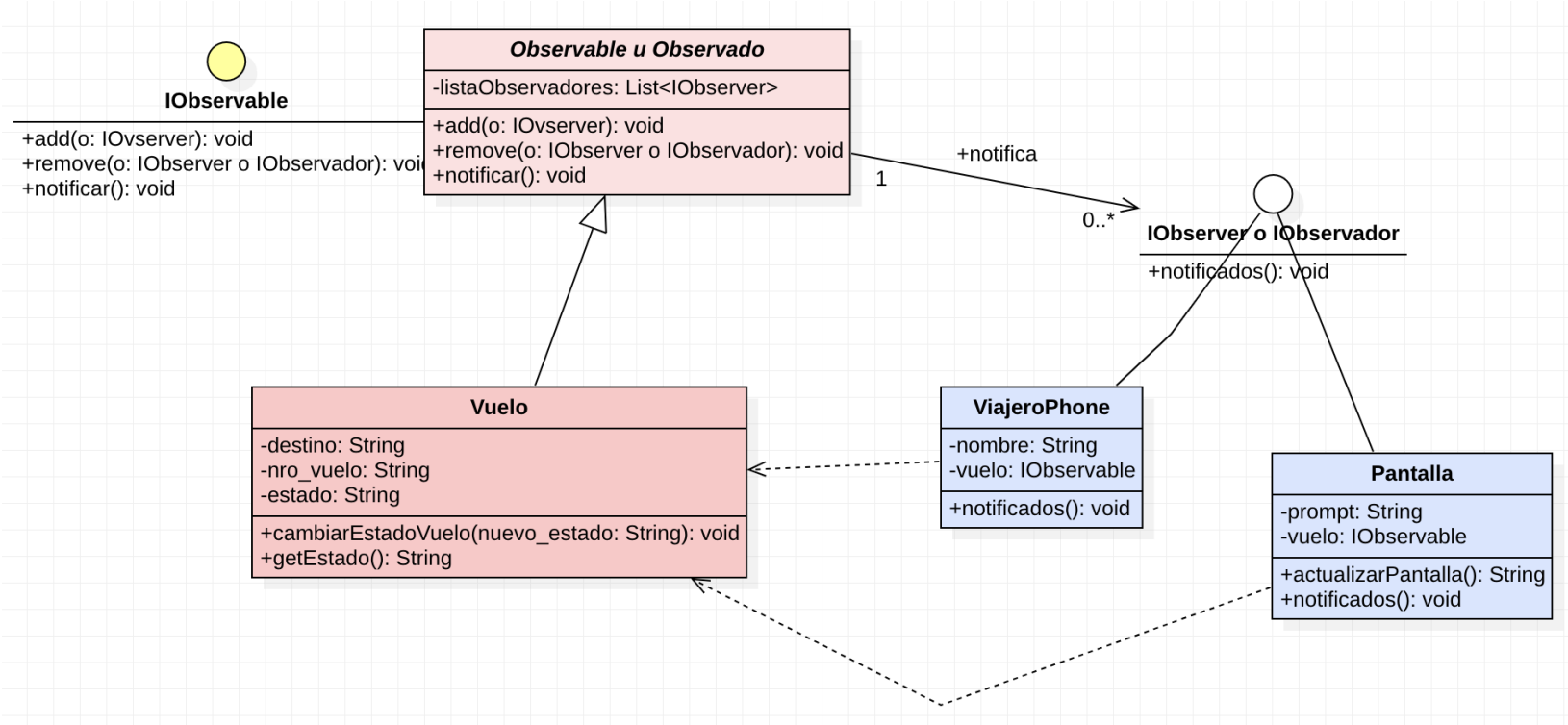
- **Interface (o clase abstracta) Observable:** Define los mensajes principales que debe entender todo sujeto que permita ser observado. El mensaje más importante se denomina "notificar", el cuál llamará a todos los observadores.
- **Sujetos concretos:** Clases que implementan (o "heredan de", si fuera clase abstracta) la interfaz Observable. Tienen el comportamiento real y eligen cuándo enviar notificaciones a sus observadores.
- **Interface Observer:** Define el método que llamarán los observables cuando estos quieran notificar sobre algo.
- **Observadores Concretos:** Clases que implementan la interfaz Observer, cuyos objetos actúan cuando los observables los llaman.

Diagrama UML

- Beneficios:
  - Mayor mantenibilidad debido a que el comportamiento por cada estado es fácilmente localizable.



Ejemplo


IObservable
+add(o: IOvserver): void
+remove(o: IObserver o IObservador): voi
+notificar(): void
Observable u Observado
-listaObservadores: List<IObserver>
+add(o: IOvserver): void
+remove(o: IObserver o IObservador): void
+notificar(): void
+notifica
1
0..*
IObserver o IObservador
+notificados(): void
Vuelo
-destino: String
-nro_vuelo: String
-estado: String
+cambiarEstadoVuelo(nuevo_estado: String): void
+getEstado(): String
ViajeroPhone
-nombre: String
-vuelo: IObservable
+notificados(): void
Pantalla
-prompt: String
-vuelo: IObservable
+actualizarPantalla(): String
+notificados(): void

Práctica

- Realizar el ejercicio de aplicación de patrones "**Garbarisi**".

Magic Café ☕ ✨

- Aplicar el patrón Observer para enviar notificaciones por distintos canales.

JAVA + JPA

OBSERVER

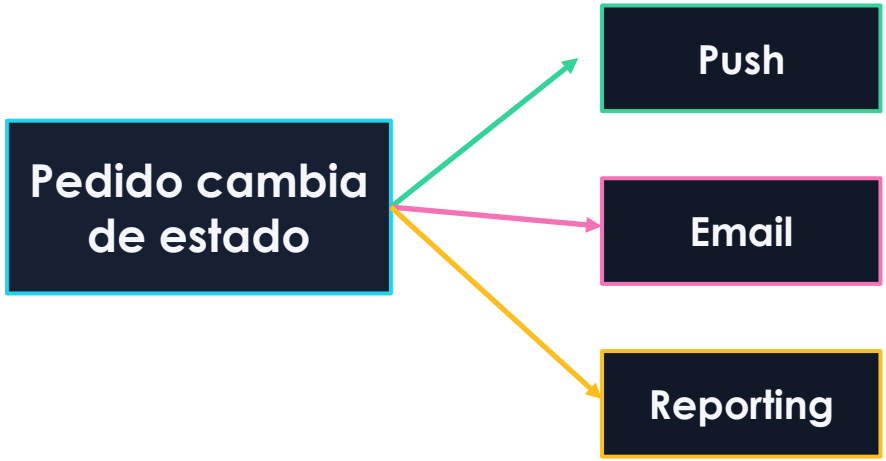


Punto de partida

- El pedido ya podía armarse, cobrarse y avanzar por estados.



- Problemas nuevo:
  - Cada vez que un pedido cambia de estado necesitamos avisar a distintos sistemas: app del cliente, email y reporting.
  - La clase Pedido no debería conocer **Firebase**, **JavaMail** ni **reportes**.



Antes y después del Observer

- La mejora es desacoplar el evento de cambio de estado de las acciones que se disparan.

ANTES

```
public void cambiarEstado(...) {
  this.estado = nuevoEstado;
  firebase.enviar(...);
  email.enviar(...);
  reporting.informar(...);
}
```

Problema: Pedido queda acoplado a integraciones externas.

OCP

DESPUÉS

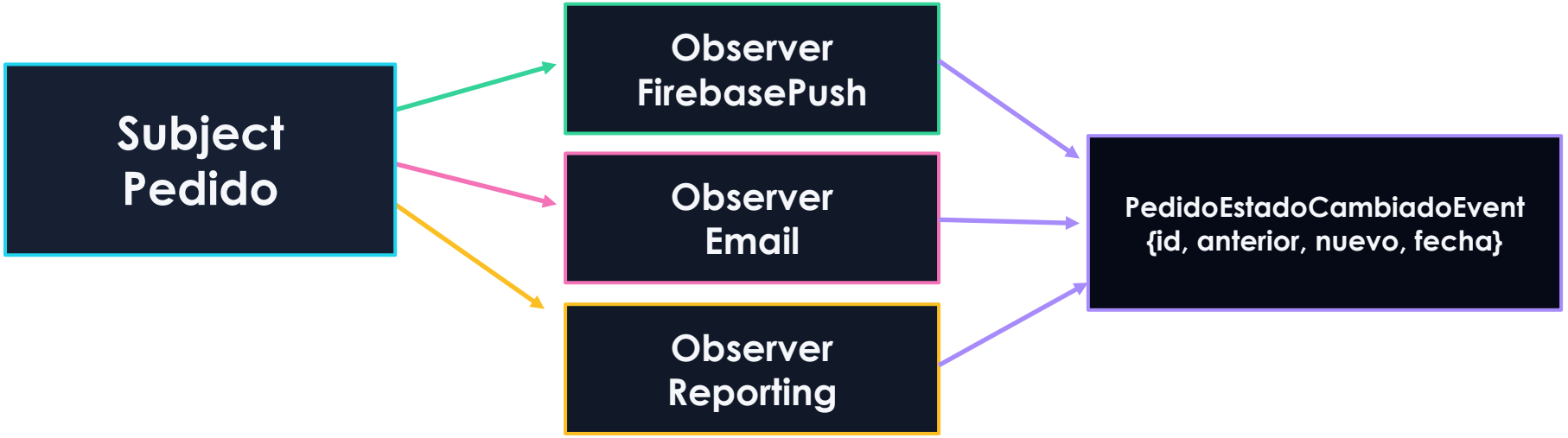
```
public void cambiarEstado(...) {
  this.estado = nuevoEstado;
  notificarObservers(evento);
}

observer.actualizar(evento);
```

Beneficio: puedo agregar más avisos sin tocar la lógica central del pedido.

Qué resuelve el patrón Observer

- Un objeto Subject avisa automáticamente a muchos Observer cuando ocurre un evento.



- El pedido solo publica "**pasó algo**". Cada observador decide qué hacer con ese evento.

Dónde vive cada pieza en Clean Architecture

- Dominio define contratos; infraestructura implementa integraciones.
- **Importante**: Pedido no importa librerías ni sabe cómo se manda un email. Solo conoce la interfaz PedidoObserver.

DOMAIN
Pedido, PedidoObserver, Event

APPLICATION
MagicCafeService

PRESENTATION
Controller + Swing

INFRASTRUCTURE
Firebase, Email, Reporting, JPA

Paso 1: crear el contrato del Observer

- Primero definimos qué mensaje reciben todos los observadores.

PedidoObserver	PedidoSubject	PedidoEstadoCambiadoEvent
actualizar(evento)	agregarObserver() quitarObserver() notificarObservers()	pedidoId estadoAnterior estadoNuevo fechaCambio

```
public interface PedidoObserver {
  void actualizar(PedidoEstadoCambiadoEvent evento);
}
```

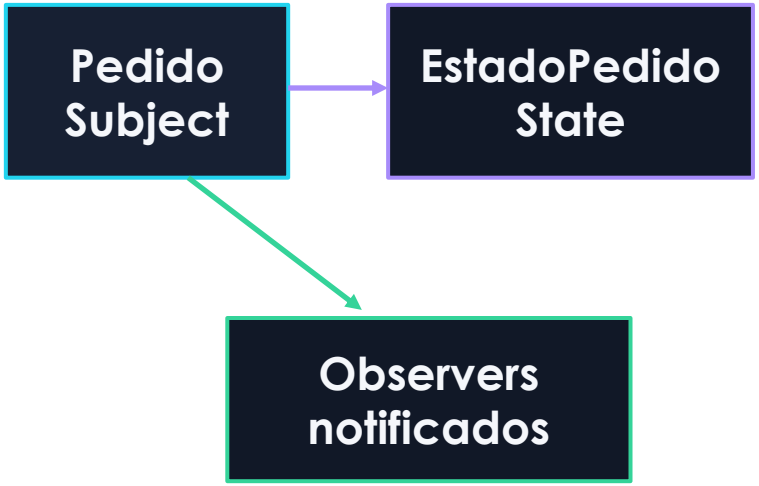
Esto nos permite tratar igual a cualquier canal: push, email, auditoría, reporting, WhatsApp, etc.

Paso 2: Pedido publica el evento al cambiar de estado

- State cambia el estado; Observer avisa que el cambio ocurrió.

```
public void cambiarEstado(EstadoPedido nuevoEstado) {
  String anterior = this.estado;
  this.estadoActual = nuevoEstado;
  this.estado = nuevoEstado.getNombre();

  if (!anterior.equals(this.estado)) {
    notificarObservers(new PedidoEstadoCambiadoEvent(
      this, anterior, this.estado));
  }
}
```

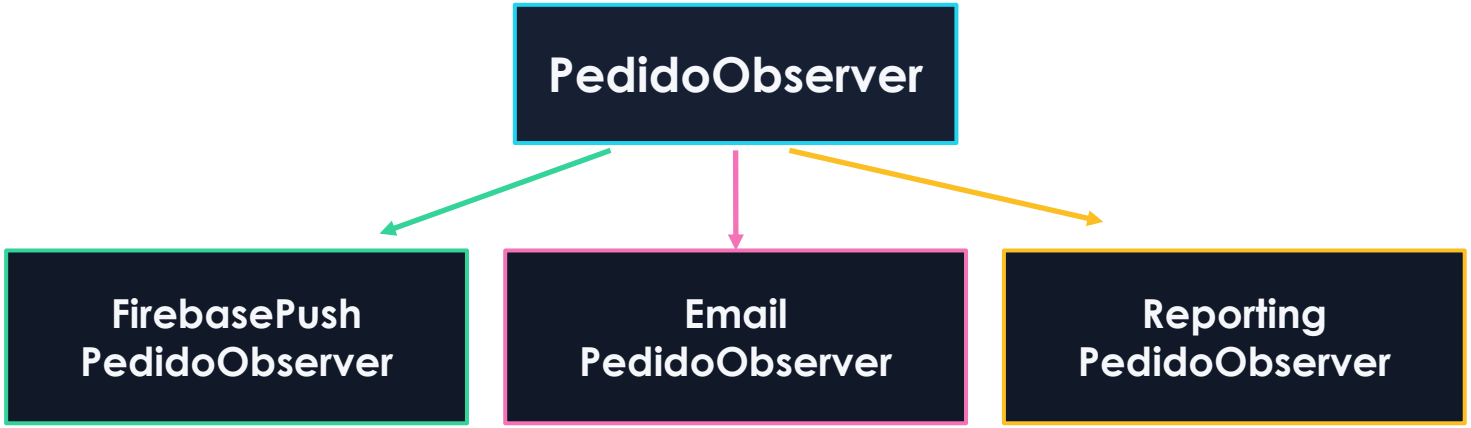


Importante para JPA

La lista de observers es **@Transient**: no se guarda en base porque representa integraciones de runtime.

Paso 3: agregar observadores concretos

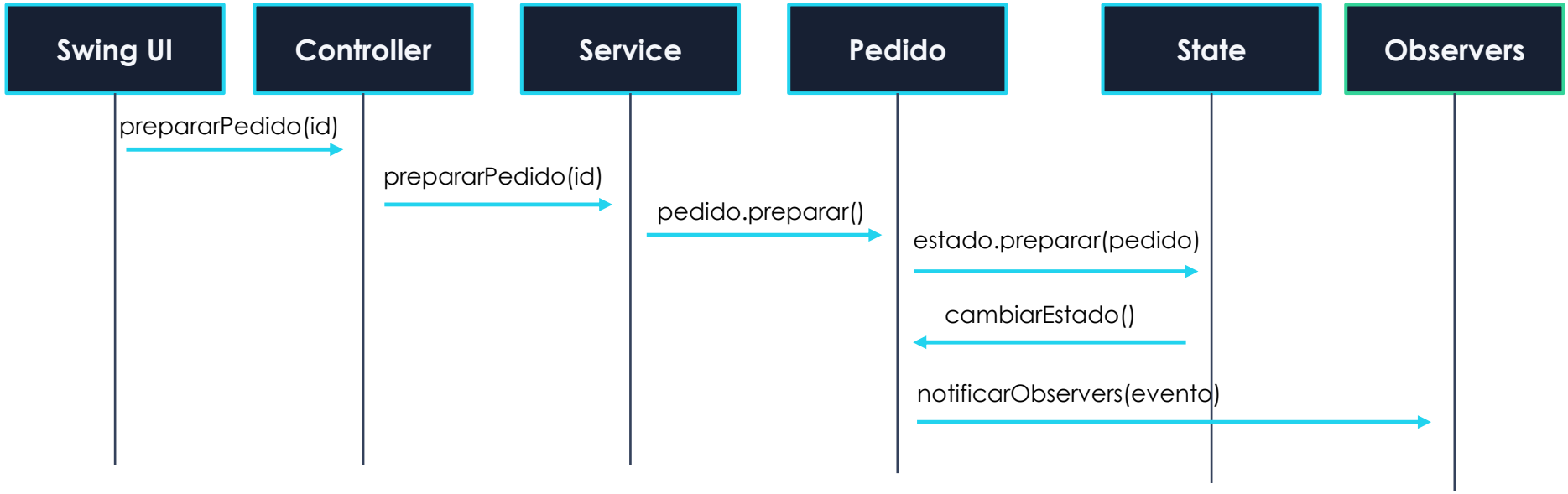
- Cada canal implementa la misma interfaz y reacciona a su manera.




```
service = new MagicCafeService(
  vendibleRepo, pedidoRepo, descuentoRepo,
  List.of(new FirebasePushPedidoObserver(),
          new EmailPedidoObserver(),
          new ReportingPedidoObserver()));
```

Flujo completo: de la UI a las notificaciones

- La UI no sabe nada de observers; solo pide avanzar el estado.



- **Resultado**: se actualiza el estado y salen los avisos sin acoplar la UI ni el Pedido a canales concretos.

Diagrama UML

- Vista UML simplificada de las clases interviniente.

Por qué queda mejor diseñado

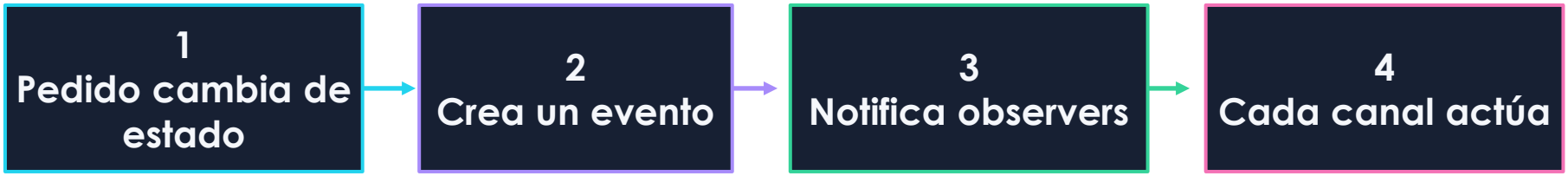
- Observer suma extensibilidad sin romper lo que ya funcionaba.

Bajo acoplamiento GRASP	Alta cohesión GRASP	Open/Closed SOLID	Dependency Inversion SOLID
Pedido no depende de Firebase, Email ni Reporting.	Pedido gestiona estados. Cada observer gestiona su canal.	Para sumar WhatsApp, agrego otro observer sin tocar Pedido.	El dominio depende de abstracciones, no de implementaciones.

```
// Nuevo canal sin modificar Pedido
public class WhatsAppPedidoObserver implements PedidoObserver {
  public void actualizar(PedidoEstadoCambiadoEvent e) {
    // enviar WhatsApp
  }
}
```

Conclusión

- Cuando el pedido cambia de estado, el dominio publica un evento. Los observadores se enteran y ejecutan acciones externas, sin que el Pedido conozca esas integraciones.



Strategy + Decorator + Composite + State + Observer

Beneficios

- **Bajo acoplamiento**: el objeto principal (Subject) no conoce detalles de quién recibe las notificaciones.
  - pedido.notificar();
- El Pedido no sabe si existe:
  - EmailNotificationObserver
  - PushNotificationObserver
  - WhatsAppNotificationObserver
  - ReportingObserver
- **Fácil de extender (Open/Closed Principle):** si mañana el negocio pide:
  - Telegram
  - SMS
  - Discord
  - Microsoft Teams
  - No modificamos Pedido.

Beneficios

- **Centraliza eventos del dominio**
  - El cambio ocurre en un solo lugar, Pedido, y desde allí se disparan todos los efectos secundarios.
- **Muy alineado con sistemas modernos**
- Lo vemos en:
  - Spring Events
  - Kafka
  - RabbitMQ
  - Azure Service Bus
  - AWS SNS
  - Domain Events (DDD)

Desventajas

- **Difícil de seguir el flujo:**

  pedido.cambiarEstado();

puede parecer simple, pero por detrás:

  - EmailObserver
  - PushObserver
  - WhatsAppObserver
  - ReportingObserver
  - AuditoriaObserver
- **Puede generar efectos inesperados:** un dev agrega "MarketingObserver" y de repente cada pedido manda campañas de marketing. El Subject no tiene control sobre eso.
- **Problemas de rendimiento:** si hay 100 observers, cada actualización dispara "observer.update();". Si además hacen llamadas externas (SMTP, Firebase, WhatsApp API), puede volverse lento.

Observer en el mundo real

- Nota: el video se muestra con JavaFX dentro de una pantalla Swing.



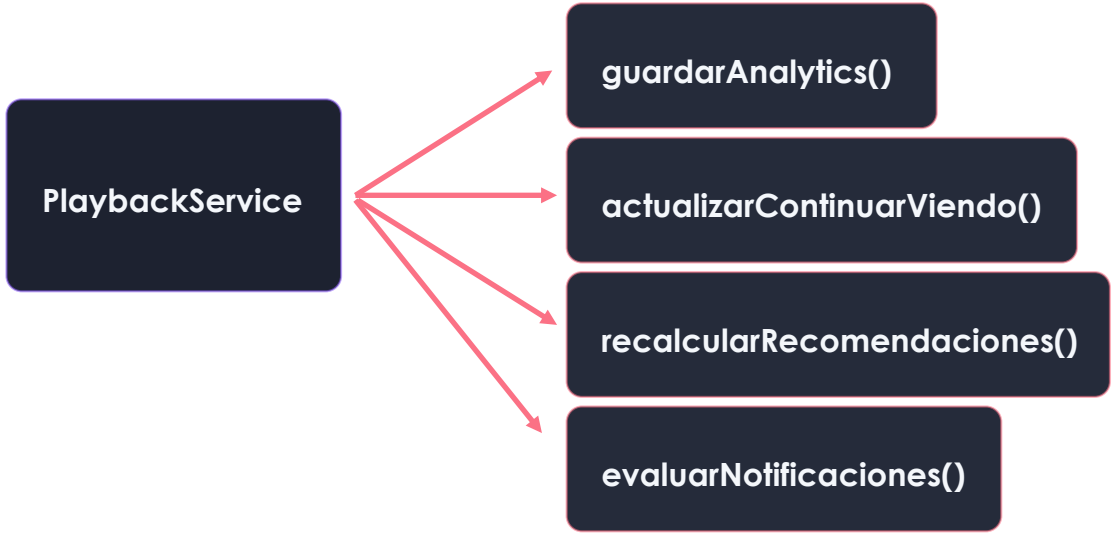
Punto de partida: Observer clásico

- Subject conoce la lista de observers, pero no conoce sus implementaciones concretas.
- Bajo acoplamiento: el Publisher no sabe qué hace cada observer.
- Open/Closed: agregamos observers sin tocar PlaybackService.
- El evento permite que varias acciones ocurran ante el mismo Play



Antes: una clase centralizada y acoplada

- Sin Observer, el servicio principal empieza a tener demasiadas responsabilidades.

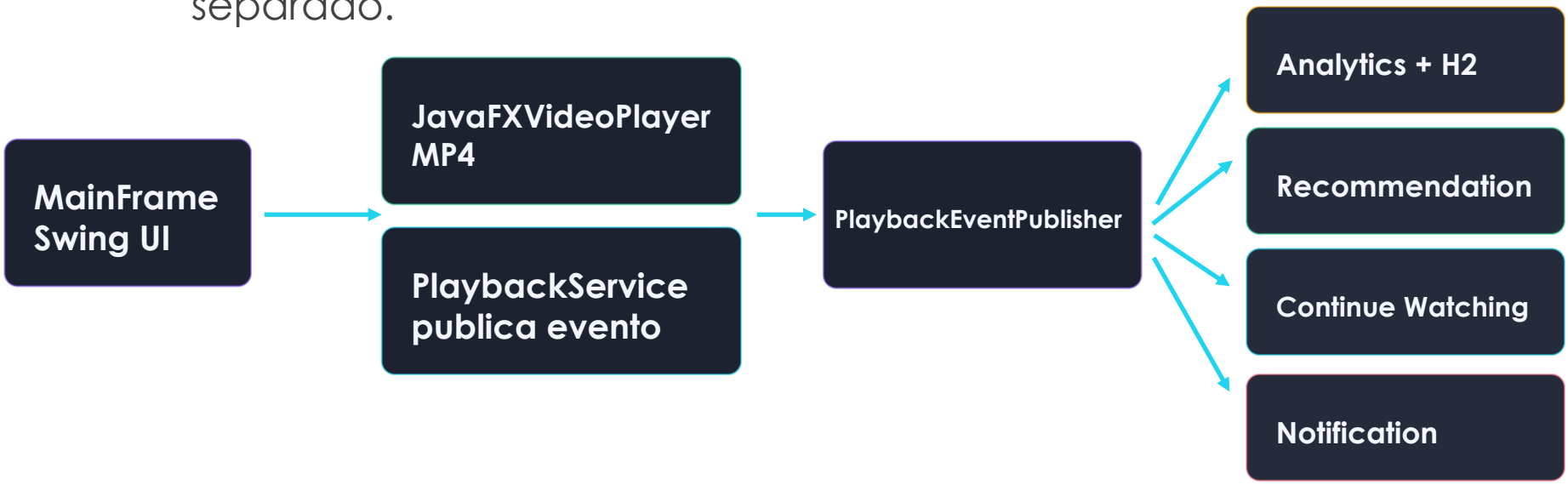


Problema

- Alta dependencia entre módulos.
- Cada nueva reacción modifica el servicio.
- Difícil testear y mantener.
- Se parece a una God Class.

Después: Play Event + Subscribers

- El servicio reproduce y publica; los observers reaccionan por separado.



Paso 1: el usuario selecciona un MP4 local

- La UI Swing usa JFXPanel para incrustar JavaFX.
- El MP4 se reproduce con Media + MediaPlayer + MediaView.

src/main/resources/videos/

```
videos/
  toystory.mp4
  mandaloriangrogu.mp4
  COLOCA_TUS_MP4_AQUI.txt
```

→

StreamingContent
resourcePath=/videos/toystory.mp4

→

JavaFX MediaPlayer

Paso 2: se publica ContentStartedEvent

- El evento representa algo que ocurrió en el dominio: comenzó una reproducción.

```
public class ContentStartedEvent {
    private final String userId;
    private final StreamingContent content;
    private final LocalDateTime occurredAt;
}
```

```
public void play(String userId, StreamingContent content) {
    ContentStartedEvent event = new
                    ContentStartedEvent(userId, content);
    publisher.notifyContentStarted(event);
}
```

Dato importante

El evento no dice “mandá email”, “guardá DB” o “recomendá”.
Solo comunica que algo pasó.

Paso 3: cada observer hace su trabajo

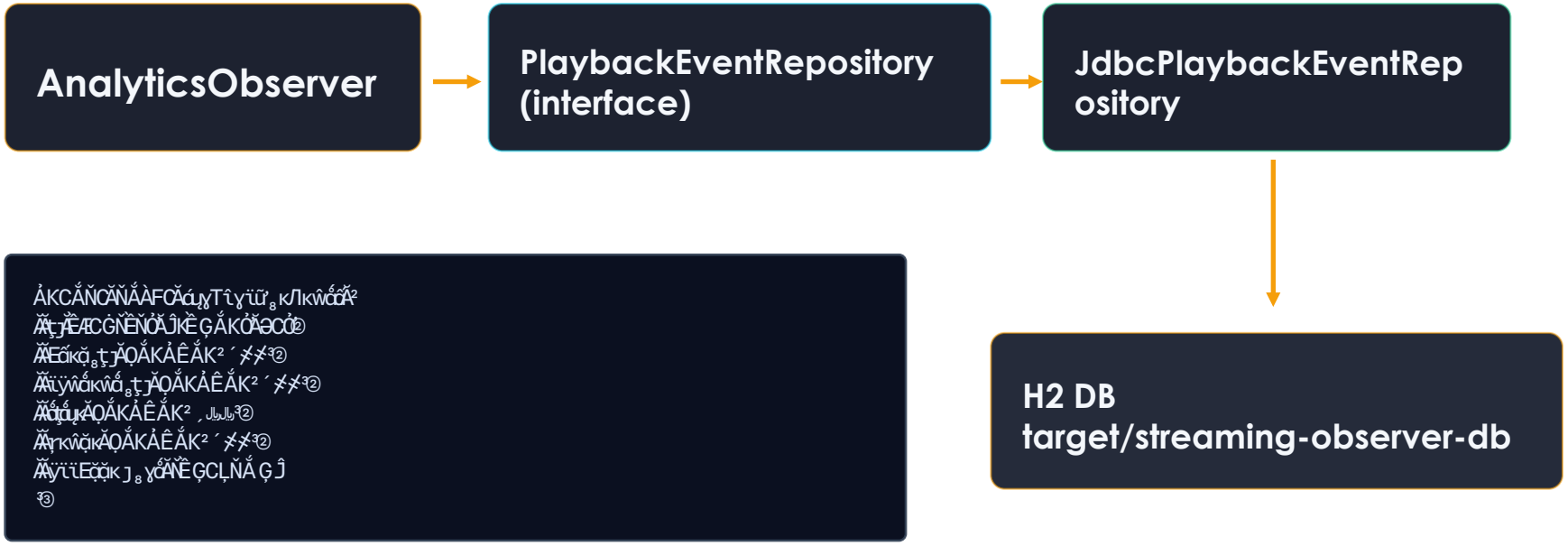
- Cada subscriber implementa la misma interfaz, pero con una reacción distinta.



- Agrego un nuevo observer sin tocar PlaybackService.
- Cada clase tiene una única responsabilidad.
- La demo se parece a cómo reaccionan servicios reales ante eventos.

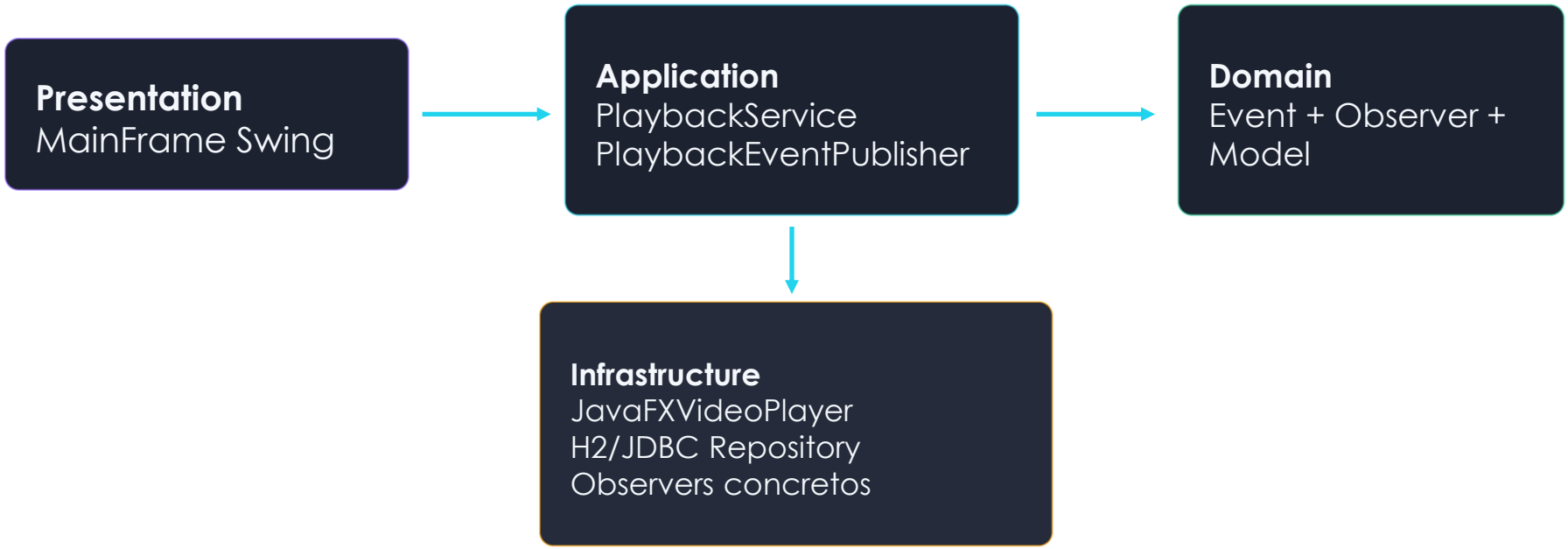
Paso 4: persistencia con H2 + JDBC

- Agregamos una base embebida para que el ejemplo tenga trazabilidad.



Arquitectura del proyecto

- El dominio define interfaces.
- Infrastructure implementa detalles técnicos.
- Main/AppFactory cablea dependencias manualmente.



De Observer a Event Driven

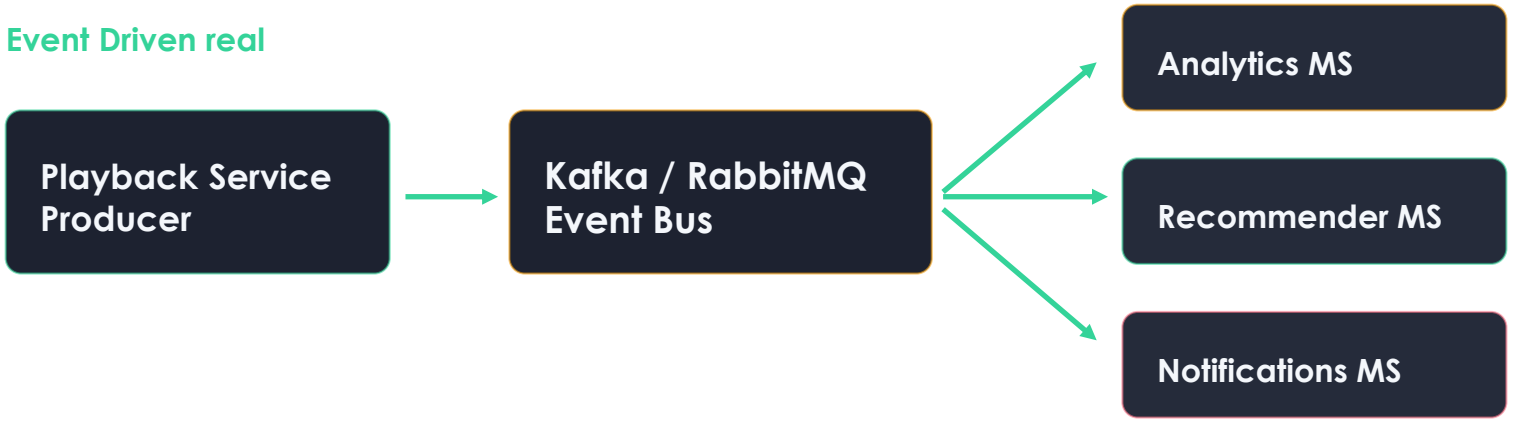
- Mismo concepto, distinta escala.

Observer académico



Nuestra demo reemplaza Kafka por una lista de observers en memoria.

Event Driven real



Conclusión

- Aplicamos **Observer** para desacoplar el servicio de reproducción de las acciones que deben ocurrir cuando un usuario inicia un contenido. **PlaybackService** publica un evento, mientras que **Analytics**, **Recommendation**, **Continue Watching** y **Notification** reaccionan como **observers independientes**.
- Esto permite agregar nuevos **subscribers** sin modificar el servicio principal, cumpliendo **Open/Closed** y **reduciendo** el **acoplamiento**.

Pros	Contras	Evolución
Extensible	Flujo menos visible	Kafka
Bajo acoplamiento	Errores distribuidos	RabbitMQ
Más cohesión por clase	En memoria no escala solo	Microservicios Event Driven

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

